Object-Oriented Programming and Data Structures

COMP2012: Lambda Function & Function Pointer

Cecia Chan
Brian Mak
Pedro Sander

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China

Part I

Local Anonymous Functions — Lambdas

Lambda Expressions (Lambdas)

Syntax: Lambda

[<capture-list>] (<parameter-list>) mutable →<return-type> { <body> }

- They are anonymous functions — functions *without* a name.
- They are usually defined locally inside functions, though global lambdas are also possible.
- The capture list (of variables) allows lambdas to use local variables that are already defined in the enclosing function.
  - [=]: capture all local variables by value.
  - [&]: capture all local variables by reference.
  - [variables]: specify only the variables to capture
  - global variables can always be used in lambdas without being captured. It is an error to capture them in lambdas.
- The return type
  - is void by default if there is no return statement.
  - is automatically inferred if there is a return statement.
  - may be explicitly specified by the → syntax.

Example: Simple Lambdas with No Captures

```
1   #include <iostream>     /* File : simple-lambdas.cpp */
2   using namespace std;
3
4   int main()
5   {
6       // A lambda for computing squares
7       int range[] =  { 2, 5, 7, 10 };
8       for (int v : range)
9           cout << [](int k) { return k * k; } (v) << endl;
10
11      // A lambda for doubling numbers
12      for (int& v : range) [](int& k) { return k *= 2; } (v);
13      for (int v : range) cout << v << "\t";
14      cout << endl;
15
16      // A lambda for computing max between 2 numbers
17      int x[3][2] = { {3, 6}, {9, 5}, {7, 1} };
18      for (int k = 0; k < sizeof(x)/sizeof(x[0]); ++k)
19          cout << [](int a, int b) { return (a > b) ? a : b; } (x[k][0], x[k][1])
20               << endl;
21
22      return 0;
23  }
```

Example: Lambdas with Captures

```
1   #include <iostream>     /* File : lambda-capture.cpp */
2   using namespace std;
3   int main()
4   {
5       int sum = 0, a = 1, b = 2, c = 3;
6
7       for (int k = 0; k < 4; ++k) // Evaluate a quadratic polynomial
8           cout << [=](int x) { return a*x*x + b*x + c; } (k) << endl;
9       cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
10
11      for (int k = 0; k < 4; ++k) // a and b are used as accumulators
12          cout << [&](int x) { a += x*x; return b += x; } (k) << endl;
13      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
14
15      for (int v : { 2, 5, 7, 10 }) // Only variable sum is captured
16          cout << [=]() mutable  { return sum += a*v; } () << endl;
17          //cout << [&sum](int x) { return sum += a*x; } (v) << endl; // Error!
18      cout << "sum = " << sum << endl;
19
20      return 0;
21  }
```

```
lambda-capture.cpp:16:47: error: variable 'a' cannot be implicitly captured
      in a lambda with no capture-default specified
        cout << [&sum](int x) { return sum += a*x; } (v) << endl;
                                                 ^
```

Example: When Are Values Captured?

```
1   #include <iostream>     /* File : lambda-value-binding.cpp */
2   using namespace std;
3
4   int main()
5   {
6       int a = 1, b = 2, c = 3;
7       auto f = [=](int x) { return a*x*x + b*x + c; };
8
9       for (int k = 0; k < 4; ++k)
10          cout << f(k) << endl;
11      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
12
13      a = 11, b = 12, c = 13;
14      for (int k = 0; k < 4; ++k)
15          cout << f(k) << endl; // Will f use the new a, b, c?
16      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
17
18      return 0;
19  }
```

- The keyword auto allows one to declare a variable without a type which will be inferred automatically by the compiler.
- WARNING: You are not allowed to use auto in this course!

Example: When Are References Captured?

```
1   #include <iostream>     /* File : lambda-ref-binding.cpp */
2   using namespace std;
3
4   int main()
5   {
6       int a = 1, b = 2, c = 3;
7       auto f = [&](int x) { a *= x; b += x; c = a + b; };
8
9       for (int k = 1; k < 3; f(k++))
10          ;
11      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
12
13      a = 11, b = 12, c = 13;
14      for (int k = 1; k < 3; f(k++)) // Will f use the new a, b, c?
15          ;
16      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
17
18      return 0;
19  }
```

Question: What is the printout now?

Capture by Value or Reference

- When a lambda expression captures variables by value, the values are captured by copying only once at the time the lambda is defined.
- Capture-by-value is similar to pass-by-value.
- Unlike PBV, variables captured by value cannot be modified inside the lambda unless you make it mutable.

Examples

```
/* File: mutable-lambda.cpp*/
int a = 1, b = 2;

cout << [a](int x) { return a += x; } (20) << endl; // Error!
cout << [b](int x) mutable { return b *= x; } (20) << endl; // OK!
cout << "a = " << a << "\tb = " << b << endl;
```

- Similarly, capture-by-reference is similar to pass-by-reference.

Example: Mutable Lambda with Return

```
1   #include <iostream>     /* File : mutable-lambda-with-return.cpp */
2   using namespace std;
3
4   int main()
5   {
6       float a = 1.6, b = 2.7, c = 3.8;
7
8       // [&, a] means all except a are captured by reference; a by value
9       auto f = [&, a](int x) mutable ->int { a *= x; b += x; return c = a+b; };
10
11      for (int k = 1; k < 3; ++k)
12          cout << "a = " << a << "\tb = " << b << "\tc = " << c
13               << "\tf(" << k << ") = " << f(k) << endl;
14
15      cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
16      return 0;
17  }
```

- One may mix the capture-default [=] or [&] with explicit variable captures as in [&, a] above.
- In this case, all variables but a are captured by reference while a is captured by value.
- But the exceptions must be given after [=] or [&].

Part II

Function Pointers

```
int square(int x) {  return x*x; }



int main( )
{
   int (*fp) = square;
   . . .
}
```

Function Pointer

- Inherited from C, C++ allows a function to be passed as argument to another function.
- Actually, we say that we pass the function pointer.
- E.g., for the template function `larger()` we talked before :

```
1  template <typename T>
2  inline const T& larger(const T& a, const T& b)
3  {
4      return (a < b) ? b : a;
5  }
```

the type of the function pointer pointing to it is:

```
inline const T& (*)(const T&, const T&)
```

Example: Function Pointer — smaller( ) and larger( )

```
#include <iostream>     /* File: fp-smaller-larger.cpp */
using namespace std;

int larger(int x, int y) { return (x > y) ? x : y; }
int smaller(int x, int y) { return (x > y) ? y : x; }

int main()
{
    int choice;
    cout << "Choice: (1 for larger; others for smaller): ";
    cin >> choice;

    int (*f)(int, int) = (choice == 1) ? larger : smaller;

    cout << (*f)(3, 5) << endl;
    return 0;
}
```

Example: Array of Function Pointers — Calculator

```
1  #include <iostream>     /* File: fp-calculator.cpp */
2  using namespace std;
3  double add(double x, double y) { return x+y; }
4  double subtract(double x, double y) { return x-y; }
5  double multiply(double x, double y) { return x*y; }
6  double divide(double x, double y) { return x/y; } // No error checking
7
8  int main()
9  {
10     double (*f[])(double x, double y) // Array of function pointers
11         = { add, subtract, multiply, divide };
12
13     int operation; double x, y;
14     cout << "Enter 0:+, 1:-, 2:*, 3:/, then 2 numbers: ";
15     while (cin >> operation >> x >> y)
16     {
17         if (operation >= 0 && operation <= 3)
18             cout << f[operation](x, y) << endl; // Call + - * /
19         cout << "Enter 0:+, 1:-, 2:*, 3:/, then 2 numbers: ";
20     }
21     return 0;
22 }
```

Example: Function Pointer Pointing To A Lambda

```
1   #include <iostream>     /* File: fp-smaller-larger-lambda.cpp */
2   using namespace std;
3
4   int main()
5   {
6       int choice;
7       cout << "Choice: (1 for larger; others for smaller): ";
8       cin >> choice;
9
10      int (*f)(int, int);
11
12      if (choice == 1)
13          f = [] (int x, int y) { return (x > y) ? x : y; };
14      else
15          f = [] (int x, int y) { return (x > y) ? y : x; };
16
17      cout << f(3, 5) << endl;
18      return 0;
19  }
```

Note: A function pointer cannot point to a lambda with captures.

Further Reading: Nested Lambda

```
1   #include <iostream>     /* File : nested-lambda.cpp */
2   using namespace std;
3
4   int main()
5   {
6       int a = 1, b = 1, c = 1;
7
8       auto f = [a, &b, &c]() mutable
9       {
10          auto g = [a, b, &c]() mutable      // Nested lambda
11          {
12              cout << a << b << c << endl;
13              a = b = c = 4;
14          };
15
16          a = b = c = 3; g();
17      };
18
19      a = b = c = 2; f();
20      cout << a << b << c << endl;
21      return 0;
22  }
```

Quiz: What if we capture b by value in f and by reference in g?